

Manual maestro de arquitectura, operación y gobernanza

Cómo funciona Cyrvanta de punta a punta: sus contenedores, sus ocho pantallas, el ciclo de vida de un incidente, la separación de funciones que impide que una sola persona decida sola, y el circuito de lecciones aprendidas que convierte lo que el SOC concluyó en contexto que reaparece cuando hace falta.

VERSIÓN DEL DOCUMENTO

v2.0 — reemplaza a v1.0 Master Enterprise

ARQUITECTURA

Monolito modular, Clean Architecture, Puertos y Adaptadores

TAXONOMÍA

MITRE ATT&CK — tácticas y técnicas

FECHA DE EMISIÓN

Agosto de 2026

AISLAMIENTO

PostgreSQL con RLS forzada por tenant

IDIOMAS

Español e inglés, nativos en toda la interfaz

CAPÍTULO 1

Qué es Cyrvanta y qué decide un humano

Cyrvanta es una plataforma de operaciones de seguridad para SOC: ingesta telemetría, correlaciona eventos dispersos en incidentes, los enriquece contra MITRE ATT&CK, propone respuestas de contención y deja registro auditable de todo. Está pensada para desplegarse on-premise y para no atar al cliente a ningún proveedor.

Lo que la distingue de un SIEM con automatización no es que use IA. Es **dónde pone el límite de la IA**: la inteligencia artificial redacta, sugiere y explica; nunca autoriza, ejecuta ni cierra. Cada acción con consecuencia sobre la infraestructura del cliente tiene un nombre y un apellido detrás.

1.1 Los seis principios que gobiernan el diseño

- **Aislamiento multitenant estricto.** Cada organización opera aislada. PostgreSQL aplica *Row Level Security* forzada: ninguna consulta ni mutación puede cruzar datos entre clientes, ni siquiera ejecutada por el dueño de las tablas. La identidad del tenant sale del contexto de seguridad autenticado, nunca del cuerpo de una petición.

- **Separación de funciones.** Quien propone no aprueba; quien resuelve un caso no lo cierra; quien escribe una lección no la revisa ni la activa. Estas reglas viven en el código y en *triggers* de la base, no en la configuración de roles. Un administrador puede reasignar permisos; no puede desactivarlas.
- **Auditoría inmutable.** Toda mutación, inicio de sesión, propuesta, aprobación o reversión genera un registro con marca de tiempo UTC, identificador de usuario, IP de origen y correlación. El historial se agrega; nunca se edita.
- **La salida de la IA es dato no confiable.** Todo lo que devuelve un modelo pasa por esquema estricto y validación determinista antes de guardarse. Si el modelo no responde, o responde mal, el sistema cae a un resultado determinista y lo deja registrado.
- **Bilingüe por construcción.** Interfaz, respuestas de la API, eventos de auditoría y notificaciones tienen español e inglés nativos. No hay texto de interfaz fuera del sistema de traducción.
- **Respuesta automática apagada por defecto.** La plataforma puede ejecutar contenciones, y de fábrica no lo hace sin firma humana.

LO QUE CYRVANTA NUNCA HACE SOLA

No cierra incidentes, no suprime alertas, no modifica puntajes de riesgo, no revoca accesos ni ejecuta contenciones sin autorización humana explícita. Ninguna función de aprendizaje altera estas reglas: lo que el sistema aprende se muestra, no se aplica.

Arquitectura de contenedores y redes

Cyrvanta se despliega con Docker Compose sobre tres redes. La red `internal` no tiene salida: es donde viven la base, la caché y el broker. La red `host-access` es la única que permite alcanzar servicios del anfitrión — en particular Ollama. La red `edge` expone únicamente el proxy inverso.

CONTENEDOR	PUERTO / RED	QUÉ HACE	QUÉ SE VE EN LA INTERFAZ
reverse-proxy	8080 · edge	Nginx: termina TLS y enruta la SPA y las peticiones REST.	Es la única puerta. Todo lo demás es interno.
frontend	80 · internal	SPA compilada en React 18, TypeScript y Vite.	Renderiza las ocho pantallas.
backend	8000 · internal + host-access	FastAPI: dominio, validación Pydantic v2, correlación, autorización.	Atiende toda acción del usuario.
postgres	5432 · internal	PostgreSQL 16 con RLS forzada. Sistema de registro autoritativo.	Usuarios, incidentes, claims, memoria, auditoría.
redis	6379 · internal	Caché de sesión y control de tasa. No es sistema de registro.	Autenticación y navegación instantáneas.
rabbitmq	5672 · internal	Broker de eventos de dominio con bandeja de entrada durable.	Absorbe ráfagas de alertas sin bloquear la pantalla.
worker	internal + host-access	Consume eventos: correlación, enriquecimiento MITRE, riesgo, explicaciones.	Puebla incidentes y explicaciones sin intervención.
scheduler	internal + host-access	Ciclo cada 15 s: despacho de playbooks, ingesta Wazuh, expiraciones, métricas y sugerencias de memoria.	Todo lo que aparece solo, aparece desde acá.
opensearch	9200 · internal · opcional	Índice de telemetría de alto volumen.	Búsqueda de evidencia en Alertas.
wazuh-manager	1514-1515 · opcional	SIEM que recibe eventos de agentes en hosts.	Puebla la consola de Alertas.
n8n	5678 · opcional	Orquestador de workflows externos.	Vínculos adicionales en Playbooks.

OLLAMA NO ES UN CONTENEDOR

El modelo corre en el anfitrión y se alcanza por URL configurable (`http://host.docker.internal:11434` en desarrollo). Nunca se empaqueta como dependencia, y se accede a través de un puerto `AIProvider`, de modo que mover la inferencia a otra máquina o a una GPU dedicada no toca la API. El modelo se configura por URL y nombre, jamás cableado en el código.

CORRECCIÓN RESPECTO DE LA V1

La v1 describía a `worker` y `scheduler` solo en la red interna. Con esa configuración declaraban el alias del anfitrión pero no tenían ruta hacia él: el nombre resolvía, la conexión fallaba, y toda explicación redactada por IA caía en silencio al texto determinista. Ambos están hoy en `host-access`.

Las ocho pantallas

El menú lateral tiene ocho entradas. Cada una se muestra solo si el rol del usuario tiene el permiso que la pantalla necesita: un rol sin `memory.read` no ve la entrada de Lecciones aprendidas, en lugar de verla y encontrarse con un error.

3.1 Resumen

Pantalla de inicio. Muestra la topología del SOC y los nodos perimetrales del laboratorio, el puntaje de riesgo global, los incidentes críticos activos, la salud de los servicios, y el **pulso operativo**: un gráfico de barras de alertas e incidentes de las últimas 24 horas, con el detalle exacto al pasar el puntero sobre cada barra.

3.2 Incidentes

La consola central. Es la pantalla que más creció respecto de la v1 y la que concentra la separación de funciones. Al abrir un incidente el analista encuentra:

- **Resumen y causa raíz** — explicación determinista de los vectores correlacionados, severidad y prioridad.
- **Matriz de claims** — qué se afirma sobre el caso, con qué tipo epistémico (hecho, inferencia, hipótesis, recomendación), qué evidencia lo sostiene y quién lo evaluó.
- **Expediente técnico** — lo que el caso debe poder mostrar antes de darse por resuelto. Ver §4.3.
- **Memoria operativa aplicable** — lecciones ya aprobadas que aplican a este caso. Solo lectura.
- **Qué resultó ser este caso** — registro de feedback inmutable sobre el desenlace real.
- **Colaboradores** — gente que trabaja en el caso sin quedar a cargo de él.
- **Proponer respuesta segura** — propone la ejecución de un playbook de contención.
- **Historial y auditoría** — auditoría y línea de tiempo fusionadas en orden cronológico, diciendo quién actuó, qué era esa persona para el caso en ese momento, qué cambió y por qué.

EL MENÚ DE ACCIONES LO DECIDE EL SERVIDOR

Los botones disponibles en un incidente no se calculan en el navegador: se piden al servidor, que responde qué puede hacer *esta* persona sobre *este* caso. Una acción que el usuario nunca podría ejecutar no se muestra; una que se muestra y luego se rechaza enseña a desconfiar de la pantalla.

3.3 Alertas

Consola de ingesta y triaje de la telemetría de Wazuh y OpenSearch. Filtros por estado (UNREVIEWED, RELEVANT, DISCARDED), severidad y orden. El detalle forense muestra IPs de origen y destino, categoría de la regla, severidad, resumen de activo e identidad, y el incidente correlacionado si existe.

3.4 Playbooks

Biblioteca de procedimientos de respuesta con versionado y estados (DRAFT, APPROVED, RETIRED). Cada definición declara su modo de gobernanza y su impacto. Permite vincular opcionalmente una acción nativa a un workflow de n8n.

3.5 Integraciones

Configuración y prueba de conectividad con Wazuh, OpenSearch, MISP, Ollama, n8n, ServiceNow y SMTP, más los adaptadores empresariales de EDR y firewall (Microsoft Defender, CrowdStrike Falcon, SentinelOne, Palo Alto, FortiGate). Cada tarjeta permite cargar credenciales y ejecutar una prueba de conexión en vivo.

3.6 Lecciones aprendidas

El circuito completo de conocimiento del SOC. Tiene capítulo propio: ver el Capítulo 5.

3.7 Auditoría

Bitácora de cumplimiento. Cada fila muestra resultado (SUCCESS / FAILURE), acción, fecha y hora UTC, email del ejecutor e IP de origen. Es la fuente para reportes de auditoría externa.

3.8 Administración

Gestión de usuarios y roles, matriz de permisos por rol, y configuración de LDAP / Active Directory (URI del servidor, Base DN, Bind DN, StartTLS) con mapeo de grupos del dominio a roles de Cyrvanta y aprovisionamiento *just-in-time*.

3.9 El menú de cuenta

Al pie del panel lateral, detrás de las iniciales del usuario: nombre, correo, **los roles que tiene la sesión**, idioma y tema. El tema ofrece tres opciones — seguir al sistema, claro, oscuro — y seguir al sistema significa reaccionar a los cambios del sistema operativo, no leerlos una vez al iniciar sesión.

CORRECCIÓN RESPECTO DE LA V1

La v1 describía nueve secciones e incluía un menú de *Llaves API*. Esa pantalla existe en el código pero **no tiene ruta**: hoy no es alcanzable desde la navegación. Se documenta como pendiente de decisión de producto, no como funcionalidad disponible.

Ciclo de vida de un incidente

4.1 Estados

```
nuevo → triado → investigando → contenido → resuelto → cerrado  
                                     ↘           ↗  
                                     \         /  
                                     \       /  
                                     \     /  
                                     \   /  
                                     \ /  
                                     \ reabierto
```

Cada transición exige un motivo. Cerrar y reabrir además exigen un motivo de al menos tres caracteres, porque son las dos decisiones que alguien va a tener que justificar después.

4.2 Quién puede qué, y por qué no alcanza con los permisos

Un permiso responde «¿esta persona puede hacer X alguna vez?». Es configuración: un administrador de tenant la edita. Una regla contextual responde «¿puede hacerlo *acá*?» y vive en el código, donde nadie la puede aflojar.

LAS TRES REGLAS QUE NO SON PERMISOS

Quien tiene el caso a cargo no puede cerrarlo. No porque le falte autoridad — un supervisor con el caso asignado conserva `incident.close` — sino porque aceptar una resolución es un juicio sobre el trabajo de otro, y nadie puede emitirlo sobre el propio.

Quien resolvió un caso no puede cerrarlo, aunque no estuviera asignado. Un incidente sin responsable puede resolverlo cualquiera que lo levante; cerrarlo después sería la misma persona diciendo que el trabajo está hecho y aceptando que lo está.

Un colaborador aporta, no decide. Puede agregar notas y evidencia. No puede dar el caso por resuelto, cerrarlo ni reasignarlo: eso no es aportar al caso, es juzgarlo, y de eso responde el responsable.

Reabrir está exento de la regla del cierre: es lo contrario de aprobar el trabajo propio, es admitir que el caso no terminó.

4.3 El expediente técnico y la compuerta de resolución

Dar un caso por resuelto es pedirle a otra persona que acepte tu trabajo. Hecho contra un expediente vacío es irrevisible: el supervisor aprueba una conclusión sin nada detrás, y meses después nadie puede reconstruir qué se hizo ni por qué.

Antes de pasar a *resuelto*, el caso debe tener:

- **Diagnóstico** — qué pasó.
- **Causa raíz**, o el registro explícito de que no pudo determinarse.
- **Resolución** — qué se hizo.
- **Una nota escrita por una persona.**
- **Al menos una evidencia vinculada.**

La pantalla muestra la lista con lo que falta, para que el analista sepa qué ir a hacer en vez de apretar un botón y que se lo nieguen. El servidor la vuelve a verificar al entrar a *resuelto* y rechaza con `409`: la persona sí tiene permiso, el caso todavía no está en estado de significar algo.

POR QUÉ LA CAUSA RAÍZ ADMITE «NO DETERMINADA»

Algunos incidentes terminan genuinamente sin causa. Exigir una igual produce causas inventadas, que es peor que un honesto «no pudo determinarse» que un revisor puede sopesar. Dejarla en blanco, en cambio, no satisface la compuerta.

El expediente vive en el registro de claims, no en columnas planas. Una columna puede guardar una causa raíz; no puede decir quién la afirmó, sobre qué evidencia, ni si alguien la evaluó — que es la razón por la que esta plataforma registra afirmaciones.

4.4 La revisión de la resolución

Cuando un caso queda en *resuelto*, la pantalla le dice al revisor **de quién es el trabajo que está por aceptar** y cuándo lo hizo, y qué significa cada salida: cerrar acepta la resolución; reabrir la rechaza, y el motivo escrito vuelve al analista y queda en el registro. Si quien mira es justamente la persona que no puede cerrar, la pantalla explica por qué en lugar de mostrar un menú vacío.

Lecciones aprendidas

Es el lugar donde el SOC convierte lo que aprendió de un caso en algo que reaparece la próxima vez. Tres pestañas: las lecciones, el registro, las métricas.

5.1 Registrar — el libro de casos evaluados

Todo empieza acá, o en el propio incidente, que es donde conviene usarlo. Un analista dice qué resultó ser el caso eligiendo de una taxonomía cerrada de ocho resultados que separa deliberadamente dos preguntas distintas:

¿LA DETECCIÓN ESTUVO BIEN?	¿LA RESPUESTA FUNCIONÓ?
Verdadero positivo	Acción efectiva
Falso positivo	Acción inefectiva
Positivo benigno	Acción parcial
No concluyente	Sin evaluar

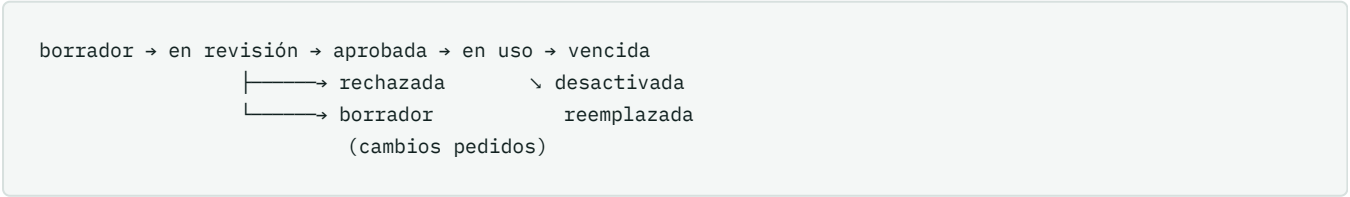
El caso se elige de una lista de incidentes y alertas reales del tenant. Los simulados no aparecen: el servidor los rechaza porque contaminarían las métricas, y ofrecer una opción que va a ser rechazada enseña a desconfiar de la pantalla.

INMUTABLE

Una entrada de feedback no se edita ni se borra. Es el cimiento de todo lo demás: si se pudiera reescribir, ninguna métrica ni lección apoyada en ella significaría nada. Es también el único activo de la plataforma que no se puede reconstruir si se pierde.

5.2 Lecciones — la base operativa

Una lección es un enunciado con **condiciones** (severidad, clasificación, estado del incidente), **evidencia** — las entradas de feedback que la sostienen — y **vigencia**. Recorre estos estados:



Una lección sin condiciones aplica a todos los incidentes mientras esté en uso, y la pantalla lo dice explícitamente al escribirla. Las clasificaciones que ofrece son las que ese tenant usa de verdad, no una lista inventada que no coincidiría con ningún caso suyo.

Corregir sin editar

Una lección no se edita: se corrige escribiendo una versión nueva. La anterior conserva su texto, sus revisiones y su historial — un enunciado aprobado que se puede reescribir es una aprobación de nada. Si la versión vigente estaba en uso, la corrección la reemplaza en el mismo acto: dejarla activa mientras su reemplazo espera revisión seguiría mostrando un consejo que su propio autor ya juzgó equivocado.

Cada versión tiene su **propio autor**, no el del candidato original, porque es a quien escribió *esa* versión a quien las reglas de separación deben impedir revisarla.

5.3 Métricas

Dos razones, calculadas sobre una ventana de 30 días:

- **Tasa de falsos positivos** — falsos positivos sobre detecciones evaluadas.
- **Efectividad de la respuesta** — acciones efectivas sobre acciones evaluadas.

Los resultados no evaluados quedan fuera del denominador: contar «nunca miramos» como «acertamos» halagaría todos los números. Cada lectura guarda su ventana, sus conteos, la versión de la definición usada y una huella de sus entradas, de modo que se puede recalcular y discutir. Por debajo de **20 casos** evaluados en la ventana, la métrica se muestra igual pero etiquetada como *muestra insuficiente*.

Son instantáneas diarias, no una consulta viva. Una métrica leída en vivo cambia bajo el lector, y dos personas mirando el mismo número el mismo día verían cosas distintas sin poder decir cuál tenía razón. Una ventana sin casos evaluados no se guarda: cero sobre cero no es una tasa.

Visible para **auditor, supervisor y administrador**. Un analista ve un mensaje que dice que no tiene permiso — no un «no hay métricas», que sería reportar una ausencia de datos donde solo hay una ausencia de permiso.

5.4 La sugerencia automática

Es la única forma de aprendizaje que la plataforma automatiza: lee lo que el SOC ya concluyó, encuentra una forma en ello y escribe un borrador para que una persona lo acepte o lo descarte.

Qué proponer lo decide un conteo, no un modelo. Cuatro o más casos juzgados igual sobre la misma clasificación son un patrón; tres son una coincidencia, y una cola llena de coincidencias es una cola que la gente deja de leer. Solo la *redacción* queda en manos del modelo, y todo lo que devuelve se valida antes de guardarse: si contesta un número, un nulo o una cadena vacía, se usa la redacción determinista.

QUÉ VE EL MODELO

Un conteo y una clasificación. **Nunca el texto que escribió el analista**, que nombra hosts, cuentas e indicadores reales: un asistente de redacción no los necesita, y lo que no se manda no se filtra. La auditoría guarda qué modelo produjo las frases, o nulo si se usó la redacción de respaldo.

Una sugerencia entra como borrador **sin autor** — no hay humano a quien excluir, así que cualquier analista puede revisarla — y no gana nada por venir de una máquina: misma cola, misma revisión humana, misma activación por una tercera persona. La tarjeta la marca, porque un revisor lee distinto una frase sabiendo que la escribió un modelo.

5.5 El interruptor de influencia

`MEMORY_INFLUENCE_ENABLED` controla **una sola cosa**: si las lecciones en uso se muestran en el incidente. Todo lo demás funciona igual con el interruptor apagado — se registra feedback, se proponen lecciones, se revisan, se activan, vencen solas, se calculan las métricas, la IA sigue sugiriendo. Lo único que no pasa es que el analista las vea cuando está trabajando el caso.

Como la influencia es solo mostrar texto ya aprobado por dos personas, encenderla no puede alterar ninguna decisión, puntaje ni acción. El interruptor existe como palanca de reversa — si alguna vez se activa una lección equivocada, se apaga sin borrar evidencia — y como compromiso previo por si la influencia creciera más allá de mostrar.

La pantalla informa el estado *de esta instalación*. Cuando las lecciones no están llegando a los incidentes, lo advierte junto al contador de las que están en uso — que es exactamente donde ese número deja de significar lo que el lector supone.

CORRECCIÓN IMPORTANTE RESPECTO DE LA V1

La v1 afirmaba que al aprobarse, una memoria «pasa a estado ACTIVE y enriquece de inmediato el motor de correlación para suprimir falsos positivos automáticamente». **Eso es falso y describe exactamente lo que la plataforma está diseñada para no hacer.** La memoria no suprime alertas, no toca el motor de correlación y no cambia ninguna decisión. Aprender de feedback sin gobierno es un canal de envenenamiento: un atacante genera ruido con una técnica hasta que el SOC la marca como falso positivo, el sistema «aprende» a bajarle prioridad, y entra por ahí.

Además, aprobar y activar son actos separados que hacen personas distintas, y el estado inicial se llama DRAFT, no PROPOSED.

Roles y permisos

El catálogo tiene **56 permisos** agrupados por área. Los cuatro roles operativos son **de sistema**: existen en cada tenant, no se pueden borrar ni renombrar, y su conjunto de permisos no se edita. Un tenant puede crear roles propios además de estos.

ROL	CÓDIGO	PERMISOS	PARA QUÉ EXISTE
Tenant Administrator	tenant-admin	56	Configuración de la organización: integraciones, usuarios, roles, LDAP. Acceso total.
Supervisor SOC	soc-supervisor	39	Acepta el trabajo de otros: cierra casos, aprueba respuestas, revisa y activa lecciones, ve métricas.
Analista SOC	soc-analyst	26	Trabaja los casos: triaje, investigación, resolución, propuesta de respuesta y de lecciones.
Auditor	auditor	21	Solo lectura, incluidas las métricas. Perfil de supervisión AGESIC / ISO 27001.

6.1 Matriz operativa

ACCIÓN	ANALISTA	SUPERVISOR	AUDITOR	ADMIN
Ver incidentes, alertas, claims	Sí	Sí	Sí	Sí
Crear y actualizar incidentes	Sí	Sí	—	Sí
Asignar responsable	—	Sí	—	Sí
Dar por resuelto	Sí	Sí	—	Sí
Cerrar o reabrir	—	Sí	—	Sí
Proponer respuesta	Sí	Sí	—	Sí
Aprobar o revocar respuesta	—	Sí	—	Sí
Registrar feedback	Sí	Sí	—	Sí
Proponer lección	Sí	Sí	—	Sí
Revisar lección	—	Sí	—	Sí
Activar o desactivar lección	—	Sí	—	Sí
Ver métricas del SOC	—	Sí	Sí	Sí

UN USUARIO PUEDE TENER MÁS DE UN ROL

La suma de permisos es la unión de los roles asignados. Las reglas contextuales del Capítulo 4 siguen aplicando por encima: acumular roles amplía lo que se puede intentar, nunca permite aprobar el trabajo propio.

CORRECCIÓN RESPECTO DE LA V1

La v1 describía tres roles (admin, analyst, auditor) y atribuía la segunda firma exclusivamente al administrador. Hoy hay cuatro roles de sistema y la segunda firma es del **Supervisor SOC**, que es el rol de operación de seguridad diseñado para aceptar el trabajo del analista. El administrador conserva el permiso, pero no es su trabajo hacerlo a diario.

Gobernanza de la respuesta

Cada playbook declara un modo de aprobación acorde a su impacto. Estos son los modos:

- **Cuatro ojos** — un analista propone, un supervisor independiente firma. Requerido para todo lo que corta acceso o aísla infraestructura.
- **Simple** — una sola firma. Notificaciones, tickets, correos.
- **Automático** — sin firma. Solo para acciones que recolectan contexto o sellan evidencia, sin alterar infraestructura.

PLAYBOOK	MODO	POR QUÉ
compromised-account	Cuatro ojos	Contención de identidad: desactiva una cuenta.
compromised-endpoint	Cuatro ojos	Aislamiento de red de un host de producción.
lateral-movement	Cuatro ojos	Interrumpe tráfico SMB/WinRM entre servidores.
privilege-escalation	Cuatro ojos	Revoca permisos de cuenta privilegiada.
ransomware-destructive	Cuatro ojos	Contención de cifrado en curso.
security-control-disabled	Cuatro ojos	Reactiva agentes ante posible evasión.
simulate-user-block	Cuatro ojos	Bloqueo de usuario en el laboratorio de demostración.
request-dual-approval	Cuatro ojos	Solicitud explícita de doble firma.
contain-and-document-incident	Simple	Contención documental sin corte de servicio.
notify-critical-incident	Simple	Notificación por SMTP, Teams o Slack.
escalation-notification	Simple	Escalamiento a guardia o responsable.
incident-report-email	Simple	Envío del informe del incidente.
create-security-ticket	Simple	Apertura de ticket ITSM.
phishing-malicious-email	Simple	Tratamiento de correo malicioso reportado.
malicious-indicator	Simple	Registro y difusión de un indicador.
automated-enrichment	Automático	Recolecta contexto; no altera infraestructura.
evidence-preservation	Automático	Sella hashes y logs en almacenamiento inmutable.

7.1 El flujo de cuatro ojos

Analista propone → Awaiting_Approval → Supervisor firma → 2/2 → Ejecución
↘ rechaza (con motivo)

Al recibir la doble autorización, el backend ejecuta la contención. Toda ejecución es reversible desde el propio incidente cuando el playbook define una acción de reversión, y la reversión también queda auditada con la atribución explícita de quién la accionó.

Quien propone no puede firmar su propia propuesta. Es la misma regla que rige el cierre de incidentes y la activación de lecciones, aplicada al plano de la respuesta.

Correlación multifuente y MITRE ATT&CK

8.1 El motor de correlación

El problema de fondo de un SOC es la fatiga de alertas: un solo ataque de fuerza bruta genera miles de entradas de log en minutos. El motor de Cyrvanta agrupa por **ventana temporal configurable** y por **entidad de dominio** (IP de origen o activo afectado), aplica un umbral de severidad mínima y un mínimo de eventos, deduplica, y unifica el resultado en un incidente correlacionado con severidad y prioridad.

Las reglas de correlación son **propias de cada tenant**. Una organización ajusta sus ventanas y umbrales sin afectar a ninguna otra.

8.2 Etiquetado MITRE

TÉCNICA	TÁCTICA	USO EN CYRVANTA
T1078	Defense Evasion / Valid Accounts	Credenciales legítimas usadas para acceso no autorizado.
T1110	Credential Access / Brute Force	Autenticación repetida contra SSH, RDP o Active Directory.
T1059	Execution / Command & Scripting	Ejecución de scripts en endpoints.
T1021	Lateral Movement / Remote Services	Desplazamiento por SMB o WinRM.
T1486	Impact / Data Encrypted for Impact	Cifrado de archivos por ransomware.
T1071	Command & Control / Application Protocol	Tráfico de C2 sobre HTTPS o DNS.

8.3 Explicaciones fundadas

El motor de exposición calcula deterministamente las tácticas involucradas y genera una explicación *fundada* que indica qué técnica neutraliza el playbook propuesto. Si hay un modelo disponible, la IA reescribe esa explicación para que se lea mejor — **sin agregar, quitar ni interpretar ningún hecho, puntaje, factor, técnica ni incertidumbre**. Si el modelo no responde, se muestra el texto determinista. El analista entiende la efectividad de la respuesta antes de pedir la segunda firma.

Qué corre solo, y cada cuánto

El scheduler ejecuta un ciclo cada **15 segundos**. El worker consume eventos a medida que llegan. Esto es todo lo que ocurre sin que nadie lo pida:

continuo	Correlación de alertas en incidentes El worker agrupa la telemetría normalizada según las reglas del tenant.
continuo	Enriquecimiento MITRE, riesgo y explicación Mapeo de técnicas, evaluación determinista de riesgo y redacción asistida.
15 s	Ingesta desde Wazuh La detección ya ocurrió en el manager; este intervalo es cuánto espera un hallazgo antes de quedar registrado en Cyrvanta.
15 s	Despacho y reconciliación de playbooks Ejecuciones pendientes y vencimiento de las que se colgaron.
15 s	Expiración de autorizaciones y de lecciones Una lección vencida deja de mostrarse dentro de los 15 s de su fecha de vigencia.
15 s	Sugerencia de lecciones Barrido del feedback de los últimos 30 días. Una sugerencia nueva aparece dentro de los 15 s del cuarto caso con el mismo veredicto y la misma clasificación.
5 min	Barrido de riesgo por entidad Relee las alarmas de la ventana; es mucho más trabajo que el resto del ciclo y no necesita hacerse cada quince segundos.
1 día	Instantánea de métricas Una lectura por definición y por tenant, con su ventana y su huella de entradas.

9.1 Los umbrales que gobiernan el comportamiento

UMBRAL	VALOR	QUÉ GOBIERNA
Patrón mínimo	4 casos	Cuántos casos iguales hacen que la IA proponga una lección.
Muestra suficiente	20 casos	A partir de cuántos casos una métrica deja de estar etiquetada como insuficiente.
Ventana de análisis	30 días	Período de las métricas y de la detección de patrones.

UMBRAL	VALOR	QUÉ GOBIERNA
Vigencia máxima	90 días	Cuánto puede durar una lección antes de tener que revalidarse.
Ciclo del scheduler	15 s	Cadencia de todo lo automático.

9.2 Costo de lo automático

La expiración usa una función SQL que devuelve solo lo vencido. Las métricas están protegidas por una comprobación diaria: en la práctica son dos conteos por tenant por ciclo. El panel de memoria del incidente hace una consulta por vista y registra la consulta una sola vez por caso y huella.

A TENER EN CUENTA AL ESCALAR

El barrido de sugerencias consulta el feedback de la ventana de 30 días en cada ciclo, por tenant, sin la guarda diaria que sí tienen las métricas. Con volúmenes chicos es trivial. Con miles de entradas y muchos tenants conviene ponerle la misma guarda: un patrón no cambia en quince segundos.

Ollama solo se invoca cuando hay un patrón nuevo o una explicación que redactar. El modelo queda residente treinta minutos, de modo que la segunda invocación no paga la carga inicial. La inferencia nunca está en el camino crítico: si no responde, el sistema sigue con su resultado determinista.

Despliegue, respaldo y escalamiento

10.1 ¿Hay que instalar todos los contenedores?

No. El núcleo — `backend`, `frontend`, `reverse-proxy`, `postgres`, `redis`, `rabbitmq`, `worker` y `scheduler` — es obligatorio. `OpenSearch`, `Wazuh Manager` y `n8n` son opcionales: si el cliente ya tiene su propio SIEM o su propio `n8n`, se deshabilitan los contenedores locales y se conecta Cyrvanta a los servidores existentes por configuración.

10.2 ¿Detecta los nodos de la red solo?

Modelo híbrido. La **telemetría se detecta sola**: cada dispositivo que envía logs al SIEM es ingestado a medida que genera eventos, y el mapa de activos se construye sin configuración nodo por nodo. La **capacidad de responder** se configura una vez por tipo de servicio: la API del firewall o del EDR en Integraciones, el URI del Active Directory en Administración.

10.3 ¿Cómo actúa detrás de un firewall sin ser bloqueada?

Cyrvanta no escanea ni ataca. Actúa por dos vías: una solicitud HTTPS REST autenticada al plano de administración del firewall — es el propio firewall quien ejecuta el bloqueo — o el canal de gestión saliente TLS que el agente EDR o SIEM ya mantiene abierto. El firewall no la bloquea porque la IP del servidor está en la lista blanca del plano de administración con tokens autorizados previamente.

10.4 ¿Qué se respalda?

Git guarda el **código y las recetas**: fuentes, `Dockerfile`, `docker-compose.yml`, configuración base y documentación. **No guarda la base viva ni la telemetría**. Los contenedores son efímeros: si uno falla, Docker lo reconstruye idéntico desde la receta.

Lo que sí se respalda son los volúmenes:

- `postgres_data` — usuarios, incidentes, claims, memoria, auditoría.
- `opensearch_data` — telemetría histórica.
- `wazuh_data` — reglas y llaves de agentes.
- `n8n_data` — credenciales y flujos.

En producción se aplica un `pg_dumpall` nocturno hacia almacenamiento externo seguro, más instantáneas periódicas del disco.

DESPLEGAR EXIGE RECONSTRUIR

Cyrvanta no tiene integración continua. Un `git push` por sí solo no cambia nada de lo que ve el usuario: hay que reconstruir la imagen y recrear el contenedor afectado.

10.5 Cuellos de botella y mitigación

PUNTO DE SATURACIÓN	SÍNTOMA	MITIGACIÓN
Ingesta y E/S de disco	Ráfagas de miles de eventos por segundo saturan los IOPS.	Separación estricta: telemetría cruda en OpenSearch, control en PostgreSQL. Rotación de índices y discos NVMe.
Tareas asíncronas	Retraso del worker ante escaneos masivos.	Réplicas horizontales del worker y deduplicación por ventana.
Inferencia de IA	Explicaciones simultáneas agotan VRAM o CPU.	Invocación asíncrona no bloqueante, GPU dedicada, y caída limpia al resultado determinista.
Conexiones a PostgreSQL	Decenas de analistas consultando en paralelo.	Pooling con PgBouncer y réplicas de lectura.

Integraciones y licenciamiento en clientes

Instalar Cyrvanta en un entorno corporativo real exige adaptarse a la infraestructura existente sin forzar licencias nuevas. Los tres escenarios que cubren casi todos los casos:

INFRAESTRUCTURA DEL CLIENTE	CÓMO SE CONECTA	¿AGENTES EN PCS?	COSTO DE LICENCIA
Ya tiene SIEM (Wazuh, Splunk, Elastic, Sentinel)	Cyrvanta consume la API REST del SIEM existente, que ya recolecta los eventos.	No	Sin costo adicional
Tiene Microsoft 365 E5 o Defender for Endpoint	Conexión HTTPS a Microsoft Graph Security API.	No	Incluido en la suscripción
Sin SIEM ni nube Microsoft	Despliegue del agente Wazuh por directiva de grupo de Active Directory.	Sí, automático por GPO	Sin costo — Wazuh es open source

11.1 Estado de los conectores empresariales

Las clases Python, los esquemas, el almacenamiento cifrado de claves, los formularios y las pruebas de conexión están desarrollados. En esta versión responden en **modo simulado seguro**. Conectar una tecnología real consiste en completar la llamada REST específica de la versión de API que tenga contratada el cliente; no requiere rehacer la arquitectura ni la interfaz.

Qué cambió respecto del manual v1

Este anexo existe para quien conozca la versión anterior. Lo que sigue ya no es cierto, o no lo era.

DECÍA LA V1	REALIDAD
Una memoria aprobada «enriquece de inmediato el motor de correlación para suprimir falsos positivos automáticamente».	Falso. La memoria solo se muestra. No toca la correlación, no suprime alertas y no cambia ninguna decisión.
Aprobar una memoria la deja activa.	Aprobar y activar son actos separados que hacen personas distintas.
El estado inicial es PROPOSED.	Es DRAFT.
Tres roles: admin, analyst, auditor.	Cuatro roles de sistema, con Supervisor SOC como firmante de la segunda autorización.
Nueve secciones de menú, incluida «Llaves API».	Ocho. La pantalla de llaves API existe en el código pero no tiene ruta.
El filtro de memoria sintética se etiqueta desde la interfaz.	La API no puede crear memoria sintética. El filtro es una condición del servidor, no un control de pantalla.
El segundo firmante del ejemplo es ldap-demo@cyrvanta.uy.	Esa cuenta es hoy Auditor, de solo lectura, y no puede firmar. El flujo de demostración usa una cuenta de supervisor.
Nada sobre expediente técnico, colaboradores, revisión de resolución, métricas del SOC ni sugerencias de la IA.	Todo eso existe y está documentado en los capítulos 4, 5 y 9.

Cyrvanta — Manual maestro de arquitectura, operación y gobernanza, versión 2.0. Documento técnico y operativo, confidencial. Los umbrales, roles, permisos y cadencias citados fueron verificados contra el código y la base de datos de la versión desplegada al momento de emisión.